

BuildSafe AI — Evaluation 1 Comprehensive Viva Study Guide

Student / Role: Member 4 (Evaluation 1) / Member D (Project Plan) | **ID:** IT23611238

Project: NYC Construction Violation Severity Prediction | **Course:** IT3051 Data Mining

1. Executive Project Scenario & Problem Formulation

The Real-World Scenario: The New York City Department of Buildings (DOB) inspects construction sites, buildings, and facades daily. Every violation cited is adjudicated through the Environmental Control Board (ECB). Violations carry distinct statutory severity levels under NYC Administrative Code Section 28-201.2: **Class 1 (Immediately Hazardous)**, **Class 2 (Major)**, and **Class 3 (Lesser)**. Severe violations require immediate stop-work orders and urgent re-inspections to prevent structural collapse or worker/pedestrian casualties. BuildSafe AI automates the predictive classification of violation severity at the point of citation issuance using location, building violation history, violation type, and narrative inspector descriptions.

Dataset Dimensions: 102,467 records x 46 raw columns from the official NYC Open Data portal. **Target Variable:** SEVERITY with 0 missing values across all records: **Class 2:** 56.22% (57,607 rows), **Class 1:** 39.69% (40,673 rows), **Class 3:** 4.09% (4,187 rows). **Machine Learning Task:** Supervised Multi-Class Classification evaluated via **Macro F1-score** and minority recall rather than misleading overall accuracy.

2. Team Work Division for Evaluation 1

The evaluation assesses individual understanding across the four defined pipeline stages:

Member	Evaluation 1 Ownership	Key Viva Defense Area
Member 1	Problem Understanding, Dataset & EDA	DOB ECB context, 3-class target, 102k rows, 56/40/4% class skew, zero target nulls.
Member 2	Data Cleaning & Quality Handling	Dropping 24 columns, handling missing text with 'UNKNOWN', disambiguating 827 duplicates via ECB ticket IDs, PII redaction.
Member 3	Feature Engineering & Encoding	Converting ISSUE_DATE to calendar components, building history creation (strictly prior dates), One-Hot Encoding.
Member 4 (You)	Final Preprocessing Pipeline & Data Leakage Prevention	Target isolation, chronological time-split, post-issuance leakage audit, text regex leakage scrubbing (Notebook 05), outlier defense, class weighting, feature selection, and ColumnTransformer pipeline (Notebook 06).

3. Member 4 Core Technical Ownership (Deep-Dive)

3.1 Chronological Time Split vs. Stratified Random Split

Implementation: The dataset was chronologically sorted on ISSUE_DATE_DT. An 80/20 chronological boundary was placed at April 14, 2026. Training partition: 81,903 rows (Jan 2025 to April 2026). Testing partition: 20,564 rows (April 2026 to Sept 2026).

Viva Defense: In municipal building safety, machine learning models are deployed sequentially -- predicting the severity of *future* violations using *past* citations. A standard random split would cause **temporal data leakage**: citations from late 2026 for building *B* would appear in the training set while predicting an earlier 2025 citation for building *B*. Because we engineer building recurrence features (PREVIOUS_VIOLATION_COUNT), a chronological split guarantees the model only learns from the past.

3.2 Dual Data Leakage Elimination

1. Post-Issuance Operational Leakage (Notebook 02): Downstream administrative fields were dropped: SERVED_DATE, HEARING_DATE, HEARING_TIME, HEARING_STATUS, ECB_VIOLATION_STATUS, CERTIFICATION_STATUS, PENALTY_IMPOSED, AMOUNT_PAID, BALANCE_DUE, and AGGRAVATED_LEVEL. These occur weeks or months after issuance; using them at prediction time is fraudulent data mining.

2. Text Target Label Leakage (Notebook 05 -- Your Standout Discovery): We audited 7,378 rows mentioning 'class' in VIOLATION_DESCRIPTION. Over 4,273 records contained direct severity phrases (e.g. 'CLASS 1 ITEMS:', 'CLASS 2'). We engineered a targeted regex (pat_severity) that scrubbed 100% of these severity numbers to zero in VIOLATION_DESCRIPTION_CLEAN while preserving 2,282 Multiple Dwelling Law (MDL) housing conversion classes ('Class A apartment') and 659 school 'classrooms'.

3.3 Outlier Profiling & Technical Justification (Notebook 06 Task 1)

Statistical Profile: All 5 calendar features have 0 outliers and near-zero skewness (-0.08 to +0.30). Building history features are right-skewed (skewness 3.96 to 10.25). 75% of citations have ≤ 2 prior violations ($Q3=2.0$), yielding an IQR upper bound of 5.0 violations. Exactly 9,630 records (9.40%) exceed 5 violations, extending up to 60.

Severity Correlation: Severe citations (Class 1 and 2) average **1.7 prior violations**, whereas minor Class 3 citations average only **0.43 prior violations**.

Viva Decision & Justification: Retain all outliers without trimming or hard capping. In NYC enforcement, extreme counts represent genuine chronic repeat-offender landlords or massive multi-unit complexes. Dropping them would discard the top 9.4% most dangerous properties in the city. Tree-based learners are invariant to monotonic tails, and linear models are normalized cleanly.

3.4 Class Imbalance Strategy (Notebook 06 Task 2)

The Imbalance: Class 3 (Lesser) is a severe minority class at only **4.11%** of training data (13.8x smaller than Class 2).
Why SMOTE was Rejected: In an 8,020-dimensional space of binary one-hot indicators and TF-IDF word frequencies, synthetic linear interpolation generates fractional categorical representations and artificial word combinations that do not correspond to any valid building violation. It also multiplies memory overhead.
Why Random Under-Sampling was Rejected: Down-sampling would throw away over 85,000 real citations.
Selected Decision: Balanced Cost-Sensitive Class Weights ($w_j = N / (K * n_j)$). Penalizes Class 3 errors by **8.16x**. On our chronological test set, balanced class weights increased **Class 3 Recall from 49.82% to 83.35%** (+33.5% surge in detecting rare citations).

3.5 Feature Relevance & Selection Analysis (Notebook 06 Task 3)

Hypothesis Testing: Evaluated all 8,020 features on training data only: ANOVA F-tests proved that building history features carry massive signal (PREVIOUS_CLASS2_COUNT $F=314.1$, $p < 10^{-130}$), whereas ISSUE_DAY ($p=0.123$) is statistically insignificant. Chi-Square tests proved that primary infraction codes ('INFRACTION_CODE1') and zoning types dominate severity prediction ($\chi^2 > 10,000$).
Dimensionality Reduction Benchmark: Keeping all 8,020 features caused optimizer convergence failures (82.0s fit time). Selecting the **Top 3,000 features via SelectKBest(χ^2 , $k=3000$)** reduced dimensionality by **62.6%**, cut training time by **60%**, and improved test Macro F1 from **86.57% to 99.55%** by eliminating sparse collinear noise.

3.6 Unified Scikit-Learn Pipeline Architecture (Notebook 06 Task 4)

Pipeline Composition: Encapsulated numerical scaling (MinMaxScaler), categorical encoding (OneHotEncoder(handle_unknown='ignore')), text vectorization (TfidfVectorizer(max_features=5000, ngrams=(1,2))), and feature selection (SelectKBest(χ^2 , $k=3000$)) into one unified Scikit-Learn Pipeline.
Zero Preprocessing Leakage: Fitted strictly on X_{train} via pipeline.fit(). The test set was evaluated strictly via pipeline.predict(), achieving **99.48% Accuracy and 99.62% Macro F1**.
Backend Integration: The entire pipeline was serialized to notebooks/models/unified_preprocessing_pipeline.pkl. A live simulation proved that a raw incoming citation is transformed and predicted as **CLASS - 1** in real time.

4. Project Files Roadmap -- What to Study for the Viva

Priority	File Path in Repository	What You Must Review & Know
#1 (Crucial)	notebooks/06_preprocessing_pipeline.ipynb	Your primary masterpiece: Task 1 Outliers (IQR, skewness, retention justification), Task 2 Class Imbalance (SMOTE vs Weights, recall surge), Task 3 Feature Selection (ANOVA, χ^2 , top 3000), Task 4 Unified Pipeline.
#2 (Crucial)	notebooks/05_text_leakage_investigation.ipynb	Your text leakage discovery: 7,378 class mentions, 4,273 severity phrases, pat_severity regex, preserving Class A/B/J and classroom context.
#3 (High)	notebooks/models/unified_preprocessing_pipeline.pkl	The serialized model artifact you built, showing how FastAPI backend will load it in Stage 9.
#4 (Medium)	notebooks/04_model_preparation.ipynb	Review Cells 3-6 showing the 80/20 chronological split boundary (April 14, 2026) and baseline algorithms.
#5 (Medium)	notebooks/02_feature_investigation.ipynb	Review the table of 24 dropped post-issuance leakage columns (penalties, hearing statuses, etc.).
#6 (Reference)	data/processed/buildsafe_text_cleaned.csv	The production dataset containing VIOLATION_DESCRIPTION_CLEAN (102,467 rows x 23 columns).

5. Top 8 Viva Questions & Word-for-Word Model Answers

Q1: What was your specific individual contribution to Evaluation 1?

I was responsible for Final Preprocessing Pipeline and Data Leakage Prevention. Specifically, I: (1) Enforced an 80/20 chronological time-split to prevent temporal leakage; (2) Audited and eliminated post-issuance leakage and discovered that 4,273 records contained literal target labels in the text narrative, which I scrubbed using a validated regex while preserving legal occupancy classes; (3) Evaluated outlier skewness and justified retaining high-risk repeat buildings; (4) Solved the 4.1% Class 3 imbalance using balanced cost-sensitive weights (+33.5% recall boost); and (5) Built and serialized a unified ColumnTransformer pipeline combining numerics, one-hot categories, and TF-IDF text.

Q2: Why did you choose a chronological time-split instead of a random stratified split?

In real-world municipal inspection, predictive systems are deployed sequentially -- predicting future violations using historical records. A random split would introduce temporal data leakage: citations from late 2026 for building B would appear in the training set while predicting an earlier 2025 citation for building B. Because we engineered historical recurrence features, a random split leaks future building history into past predictions. Our time split at April 14, 2026 guarantees 100% realistic, leakage-free evaluation.

Q3: What is the difference between target leakage and preprocessing leakage, and how did you prevent both?

Target leakage occurs when an input feature contains information that is only generated after the target event occurs, or contains the target label itself. We prevented this by dropping post-issuance fields (penalties, hearing dates) and scrubbing explicit severity tokens from text descriptions. Preprocessing leakage occurs when test set statistics influence training transformations. We prevented this by fitting all transformers -- MinMaxScaler, OneHotEncoder, and TfidfVectorizer -- strictly on the training split using `fit_transform()`, and applying only `transform()` on the test split.

Q4: Why didn't you just remove the word 'class' entirely from the description text?

A naive blanket removal of 'class' would destroy critical domain features. Over 2,280 citations reference Multiple Dwelling Law (MDL) occupancy classifications regarding illegal residential conversions, explicitly termed 'Class A apartments' or 'Class B rooming houses'. Over 650 citations describe school facility 'classrooms'. Removing 'class' wholesale blinds the model to illegal room conversions. Our regex specifically targets 'class' followed by numbers 1, 2, 3, Roman numerals, or spelled-out words, removing 100% of target labels while preserving 100% of occupancy and classroom features.

Q5: Did you detect outliers in your numerical features, and why didn't you drop them?

Yes. All calendar features had 0 outliers. In contrast, building recurrence features were right-skewed with an IQR upper bound of 5.0 violations; 9,630 records (9.40%) exceeded 5 violations, extending up to 60. We retained all outliers without trimming because in municipal code enforcement, repeat violations represent authentic chronic non-compliant landlords and large multi-unit complexes, not data entry noise. Dropping them would delete the top 9.4% most dangerous properties in NYC. Tree models are invariant to monotonic tails, and linear models are normalized cleanly.

Q6: How did you handle class imbalance, and why was SMOTE rejected?

*Class 3 represents only 4.11% of the data. SMOTE was rejected because our feature space is an 8,020-dimensional sparse matrix of binary one-hot dummies and TF-IDF word frequencies. Synthesizing artificial instances via k-NN interpolation creates fractional binary flags and synthetic word combinations that do not represent real violation text, while multiplying memory footprint. Instead, we applied balanced cost-sensitive class weights ($w_j = N / (K * n_j)$), penalizing Class 3 errors by 8.16x. On our test set, this increased Class 3 Recall from 49.8% to 83.4% without synthesizing fake data.*

Q7: How did you justify feature selection, and what was the impact?

We ran ANOVA F-tests on numerics and Chi-Square tests on categoricals and scrubbed text. Building recurrence features ($F > 199$ to 314) and primary infraction codes ($\text{Chi}^2 > 10,000$) carried massive signal, whereas calendar day of the month was statistically insignificant ($p=0.123$). By selecting the Top 3,000 features via `SelectKBest(chi2, k=3000)`, we reduced dimensionality by 62.6%, cut training time by 60%, and improved test Macro F1 to 99.55% by eliminating low-variance sparse collinear noise that was causing convergence issues.

Q8: How is your final preprocessing pipeline architected and deployed?

We built a formal Scikit-Learn `ColumnTransformer` that scales numerics via `MinMaxScaler`, encodes categoricals via `OneHotEncoder(handle_unknown='ignore')`, and vectorizes scrubbed text via `TfidfVectorizer(max_features=5000, ngrams=(1,2))`. This is chained with `SelectKBest(chi2, k=3000)` and `LinearSVC` into a single `Pipeline` object fitted on `X_train`. The entire pipeline is serialized to 'unified_preprocessing_pipeline.pkl'. In our live simulation, passing a raw dictionary representing an unencoded citation directly into `pipeline.predict()` instantly outputs 'CLASS - 1'.